

King Saud University
College of Computer and Information Sciences

Department of Computer Science

CSC 212 Data Structures Project Report – 2nd Semester 2024-2025

Developing a Photo Management Application

Authors

Name	ID	List of all methods implemented by each student
Amina Abdelkareem	444204580	Node,LinkedList,BSTNode,BST,Photo Classes
Maha AL Dakhil	444202648	PhotoManager & invIndexPhotoManager Classes
Raghad Alessa	444201099	Album & InvAlbum Classes

1. Introduction

This project focus on implementing a photo management system that efficiently organizes and retrieves images based on specific tags-based conditions. Users can create albums that automatically group photos matching specific criteria by utilizing tags associated with each photo. To enhance performance, inverted index is used. the inverted index will store all the tags, and each tag will point to all the photos that contain it and is had been implemented using the Binary search tree (BST).

In the report we include the following:

- **Specification:** Outlines linkedList class and BST class; as they are the Abstract Data Types (ADTs) used in this project, by writing the specification of each method in each class.
- **Design:** the conceptual details **of the main classes is presented through illustrative diagrams.** making it accessible to both technical and non-technical audiences.
- **Implementation:** analysis and compares the Big-Oh for the search methods, Album.getPhotos() and InvAlbum.getPhotos() after using the inverted index
- **Performance analysis:**

2. Specification

ADT linked list:

Elements: The elements are of generic type.

Structure: the elements are linearly arranged.

Domain: the number of elements the domain is limited only by the capacity of the memory. Each element in the linked list is typically represented as a **node**, which contains two parts:

- The **data** (or value) stored in the node.
- A **reference/pointer** to the next node in the list.

Operations:

1. Method findFirst ()

requires: list L is not empty. input: none

results: first element set as the current element. output: none.

2. Method findNext ()

requires: list L is not empty. Current is not last. input: none

results: element following the current element is made the current element.

output: none.

3.Method retrieve (Type e)

requires: list L is not empty. input: none

results: current element is copied into e.

output: element e.

4.Method update (Type e).

requires: list L is not empty. input: e.

results: the element e is copied into the current node.

output: none.

5.Method insert (Type e).

requires: list L is not full. input: e.

results: a new node containing element e is created and inserted after the current element in the list. The new element e is made the current element. If the list is empty e is also made the head element.

output: none

6.Method remove ()

requires: list L is not empty. input: none

results: the current element is removed from the list. If the resulting list is empty current is set to NULL. If successor of the deleted element exists it is made the new current element otherwise first element is made the new current element.

output: none.

7.Method full (boolean flag) input: none.

results: if the number of elements in L has reached the maximum number allowed then flag is set to true otherwise false.

output: flag

8.Method empty (boolean flag). input: none.

results: if the number of elements in L is zero, then flag is set to true otherwise false.

Output: flag.

9.Method last (boolean flag). input: none.

requires: L is not empty. Results: if the last element is the current element, then flag is set to true otherwise false.

Output: flag

ADT Binary search tree (BST):

Elements: the elements are nodes (BSTNode), each node contains the following data type: Type,Key and has LeftChild and RightChild references.

Structure: hierarchical structure; each node can have two children: left or right child; there is a root node and a current node. If N is any node in the tree, nodes in the left subtree $< N$ and nodes in the right subtree $> N$.

Domain: the number of nodes in a BST is bounded; type/class name is BST

Operations:

1.Method FindKey (int tkey, boolean found).

requires: none. input: tkey.

results: If bst contains a node whose key value is tkey, then that node is made the current node and found is set to true; otherwise found is set to false and either the tree is empty or the current node is the node to which the node with key = tkey would be attached as a child if it were added to the BST.
output: found

2.MethodInsert (int k, Type e, boolean inserted)

requires: Full (bst) is false. input: key, e.

results: if bst does not contain k then inserted is set to true and node with k and e is inserted and made the current element; otherwise inserted is set to false and 3. current value does not change. output: inserted.

3.Method Remove_Key (int tkey, boolean removed)

//requires :none.//input: tkey

results: Node with key value tkey is removed from the bst and removed set to true. If BST is not empty then root is made the current.

output: removed

4.MethodTraverse (Order ord)

requires: Binary Tree (BT) is not empty. input: ord.

results: Each element in the tree is processed exactly once by a user supplied procedure. The order in which nodes are processed depends on the value of ord (Order = {preorder inorder,postorder })

pretorder : each node processed before any node in either of its subtrees.

inorder : each node is processed after all its nodes in its left subtree and before any node in its right subtree.

postorder : each node is processed after all its nodes in both of its subtrees.

output: none

6. Method Retrieve (Type e)

Procedure Retrieve (Type e) requires: Binary tree is not empty. input: none

results: element in the current node is copied into e.

output: e.

7. MethodEmpty (boolean empty)

requires: None. input: none

results: If Binary tree is empty then empty is true; otherwise, empty is false.

output: empty

8. Method Full (boolean full)

requires: None. input: None.

results: if the binary tree is full then full is true otherwise false.

output: full

9.getRoot (BSTNode<Type> n)

requires: None. input: None.

results: returns the root of the tree, null in case the tree is empty

output: n

3. Design



4. Implementation

1. PhotoManager Class:

- **public void addPhoto(Photo p):**

This method is used to add a photo in a Linked List attribute that stores photos, it prevents inserting an already existing photo path by **isPhotoExist(Photo p)**, Also if the List is empty, it will insert it normally, see Figure.1.

```
public void addPhoto(Photo p){  
  
    if(isPhotoExist(p))  
        System.out.println("\nAdd photo is rejected, Given Photo is Duplicated");  
    else  
        photosList.insert(p);  
}
```

Figure.1: add Photo method in Photo Manager class.

- **public void deletePhoto(String path):**

This method deletes a photo from a Linked List attribute that stores photos by a given photo path, it first checks if the List is empty or not, if it's empty print a message to the user that the photo list is empty , otherwise it will traverses the list and compare each photo's path with the given path, if it's found in the list then photo will be removes and prints a success message to the user, if the photo path was not found in the list, it will just prints a failure message, see Figure.2.

```

public void deletePhoto(String path){

    if(photosList.empty()) {
        System.out.println("\nEmpty Photo List\n");
    }
    else{
        boolean isFound = false;
        photosList.findFirst();
        while(!photosList.last()) {

            if (photosList.retrieve().getPath().equals(path)){
                isFound = true;
                break;
            }//end if
            photosList.findNext();

        }//end while

        if (photosList.retrieve().getPath().equals(path))
            isFound = true;

        if(isFound) {
            photosList.remove();
            System.out.println("\nPhoto is removed Successfully\n");
        }else{
            System.out.println("\nRemoved is failed , Path: \""+ path +"\" is Invalid\n");
        }
    }
}

```

Figure.2: delete Photo method in Photo Manager class.

2. InvIndexPhotoManager Class:

- **public void addPhoto(Photo p):**

This method adds in both **photoList** (Linked List of photos) and **invIndex** (BST indexed by tags), The purpose of using the photoList is to prevent the duplication in **invIndex** because every addition for a photo it will be also added to the invIndex directly, so it first checks if the photo exists in the list, if not it will insert the photo in the photo list and in the invIndex, by retrieving the tags List from the given photo and use each tag to check if this tag is existing key in invIndex it will create a new key in invIndex with a new photolist as the data for it and inserting the given photo in it, Otherwise if key exists in invIndex then it will only insert into the data (inner Linked list of photos for BSTnode), see Figure.3.


```

public void addPhoto(Photo p){
    if(isPhotoExist(p)) { //here is to reject any duplication when addition
        System.out.println("\nAdd photo is rejected, Given Photo is Duplicated");
    }
    else{
        photosList.insert(p);

        LinkedList<String>tagsList = p.getTags();
        tagsList.findFirst();
        while(true){

            String tag1 = tagsList.retrieve();
            boolean isFound = invIndex.findKey(tag1);
            if(!isFound){
                LinkedList<Photo> PList = new LinkedList<Photo>();
                PList.insert(p);
                invIndex.insert(tag1 , PList);
            }
            else
                invIndex.retrieve().insert(p);

            if(!tagsList.last()){
                tagsList.findNext();
            }
            else
                break;

        }//end while
    }//end outer else
}

```

Figure.3: add Photo method in inverted index Photo Manager class.

- **public void deletePhoto(String path):**

This method removes the photo from both the **photoList** (the Linked List of all photos) and the **invIndex** (the BST indexed by tags), since photosList and invIndex are synchronized during addition and deletion, then photoList is reliable to determine what photo exist or not in the invIndex, When deleting a photo, photoList purpose is to quickly find the exact photo by its unique path and retrieve its tags, so It first checks if the photoList is empty and exits if there are no photos to delete, If the list is not empty, the method traverses it to find a photo matching the given path. Once found, it retrieves the photo's tags and removes the photo from the photoList, If no photo is found, a message is printed and the method exits, After removing the photo, the method iterates through the tags to update the invIndex: for each tag, it finds the corresponding list of photos and removes the deleted photo using the helper method deleteInnerPhotoListNode, If any tag's photo list becomes empty after removal, the tag itself is deleted from the BST, see Figure.4.

```

public void deletePhoto(String path){
    if(photosList.empty()) {
        System.out.println("\nEmpty Photo List\n");
        return;
    }

    LinkedList<String> tagsList = null;
    photosList.findFirst();
    while (true) {
        if (photosList.retrieve().getPath().equals(path)) {
            tagsList = photosList.retrieve().getTags();
            photosList.remove();
            break;
        }
        if (photosList.last())
            break;
        else
            photosList.findNext();
    }
    if(tagsList == null) {
        System.out.println("\nRemoved is failed , Path: \"" + path + "\" is Invalid/Not found\n");
        return;
    }
    Photo photo1 = new Photo(path,tagsList);
    tagsList.findFirst();
    boolean isFound;
    while(!tagsList.last()){
        isFound = invIndex.findKey(tagsList.retrieve());
        if(isFound) {
            deleteInnerPhotoListNode(invIndex.retrieve() , photo1);
            if (invIndex.retrieve().empty())//if the inner Linked list of the Node is actually empty
                invIndex.removeKey(tagsList.retrieve());
        }
        tagsList.findNext();
    }
    isFound = invIndex.findKey(tagsList.retrieve());
    if(isFound) {
        deleteInnerPhotoListNode(invIndex.retrieve() , photo1);
        if (invIndex.retrieve().empty())
            invIndex.removeKey(tagsList.retrieve());
    }
}
}

```

Figure.4: delete Photo method in inverted index Photo Manager class.

- **public void deleteInnerPhotoListNode(LinkedList<Photo> innerList , Photo photo):**

This helper method is used to remove a specific photo from a LinkedList<Photo>, where the list represents all photos associated with a certain tag inside the invIndex (BST), It is called when you are deleting a photo and you need to clean up its entry under each of its tags, see Figure.5.

```
private void deleteInnerPhotoListNode(LinkedList<Photo> innerList, Photo p) {  
    if (innerList.empty()) return;  
  
    innerList.findFirst();  
  
    while(true) {  
        if (innerList.retrieve().getPath().equals(p.getPath())) {  
            innerList.remove();  
            break;  
        }  
  
        if (innerList.last())  
            break;  
        innerList.findNext();  
    }  
}
```

Figure.5: delete inner photo list method in inverted index Photo Manager class.

3. Album Class:

- **public LinkedList<Photo> getPhotos():**

It starts by retrieving all photos in a linked list, the string of conditions separated by AND are split into an array, then for each photo in the list it compares the tags associated with the photo to the conditions. If a photo does not satisfy the conditions, it is removed from the list. After filtering all photos based on the conditions, the method returns the final list of photos that meet all criteria, see Figure.6.

```

public LinkedList<Photo> getPhotos() {

    LinkedList<Photo> Rphotos = new LinkedList<Photo>();

    LinkedList<Photo> photos1 = Manager.getPhotos();
    if (!photos1.empty()) {
        photos1.findFirst();
        while (!photos1.last()) {
            Rphotos.insert(new Photo(photos1.retrieve().getPath(), photos1.retrieve().getTags()));
            photos1.findNext();
        }
        Rphotos.insert(new Photo(photos1.retrieve().getPath(), photos1.retrieve().getTags()));
    }

    NumComps = 0;
    if (this.Condition.compareTo("") != 0) {
        String[] Array = Condition.split(" AND ");

        Rphotos.findFirst();
        while (!Rphotos.last()) {
            Photo photo = Rphotos.retrieve();
            if (!isTagAvailable(photo.getTags(), Array))
                Rphotos.remove();
            else
                Rphotos.findNext();
        }
        Photo photo11 = Rphotos.retrieve();
        if (!isTagAvailable(photo11.getTags(), Array))
            Rphotos.remove();
        else
            Rphotos.findNext();
    }
    return Rphotos;
}

```

Figure.6: getPhotos method in Album class.

4.InvAlbum class:

- **public LinkedList<Photo> getPhotos():**

The method starts by retrieving a binary search tree (BST) that maps unique tags to their respective linked lists of photos. If no condition is specified, it calls getAllPhotos, which traverses the entire BST and collects all photos into a single linked list. If a condition is provided, it splits the condition string into an array of tags, then it looks for the tag in the BST (findKey).

If the tag isn't found, it immediately returns an empty list. If the tag is found, the associated linked list of photos is retrieved. If it's the first tag, all photos in the list are copied into the result list (Rphotos). Otherwise, it performs an intersection between the current result (Rphotos) and the linked list of the current tag and then returns the resulting list, see Figure 7.

```
public LinkedList<Photo> getPhotos() {
    BST<LinkedList<Photo>> photosBST = invmanager.getPhotos();
    LinkedList<Photo> Rphotos = new LinkedList<Photo>();
    NbComps = 0;

    String[] tags;
    if (!condition.equals("")) {
        tags = condition.split("\\s*AND\\s*");
    } else {
        return getAllPhotos(photosBST); // No condition → retrieve all photos
    }

    for (int i = 0; i < tags.length; i++) {
        if (photosBST.findKey(tags[i])) {
            LinkedList<Photo> currentList = photosBST.retrieve();

            if (i == 0) {
                currentList.findFirst();
                while (!currentList.last()) {
                    Rphotos.insert(currentList.retrieve());
                    currentList.findNext();
                }
                Rphotos.insert(currentList.retrieve()); // Handle last element
            } else {
                Rphotos = intersectPhotoLists(Rphotos, currentList);
            }
        } else {
            return new LinkedList<Photo>(); // Tag not found → return empty list
        }
    }

    return Rphotos;
}
```

Figure.7: getPhotos method in invAlbum class.

- **private LinkedList<Photo> intersectPhotoLists(LinkedList<Photo> list1, LinkedList<Photo> list2):**

First, it initializes a new empty list where the common images of list1 and list2 will be stored.

It makes a check whether list2 is empty or not; if empty, it directly returns the result as empty.

Else, it iterates each image in list2 and it looks for that image in list1.

In the process of searching, for each comparison executed, regardless of the outcome, NbComps is incremented with each comparison performed.

If a matching image is found, it is appended to the result list.

When all images are processed, the method returns the result list containing only the common images of the two lists. See Figure 8, Figure 9.

```
private LinkedList<Photo> intersectPhotoLists(LinkedList<Photo> list1, LinkedList<Photo> list2) {
    LinkedList<Photo> result = new LinkedList<Photo>();

    if (!list2.empty()) {
        list2.findFirst();
        while (!list2.last()) {
            Photo p2 = list2.retrieve();

            if (!list1.empty()) {
                list1.findFirst();
                boolean found = false;

                while (!list1.last() && !found) {
                    NbComps++;
                    if (p2.getPath().equalsIgnoreCase(list1.retrieve().getPath())) {
                        found = true;
                    }
                    list1.findNext();
                }

                NbComps++;
                if (!found && p2.getPath().equalsIgnoreCase(list1.retrieve().getPath())) {
                    found = true;
                }

                if (found) {
                    result.insert(p2);
                }
            }
            list2.findNext();
        }
    }
}
```

Figure.8: intersectPhotoLists method in invAlbum class.

```

// Handle last element in list2
Photo p2 = list2.retrieve();
if (!list1.empty()) {
    list1.findFirst();
    boolean found = false;

    while (!list1.last() && !found) {
        NbComps++;
        if (p2.getPath().equalsIgnoreCase(list1.retrieve().getPath())) {
            found = true;
        }
        list1.findNext();
    }

    NbComps++;
    if (!found && p2.getPath().equalsIgnoreCase(list1.retrieve().getPath())) {
        found = true;
    }

    if (found) {
        result.insert(p2);
    }
}

return result;
}

```

Figure.9: intersectPhotoLists method in invAlbum class.

5. Performance analysis

- Album.getPhotos():

In this method each condition in the list of conditions is compared to the associated tags with each photo; so, the time complexity is $O(n*c*t)$

n is the size of photo linkedList

c is number of conditions tags

t is the number of associated tags with the photo

InvAlbum.getPhotos():

For each tag It looks for the tag in the BST by using findKey(), then the photos associated with this tag compared with the photos associated with the previous tag Photos that exist in both lists are added to the result list; so the time complexity is $O(t*\log(K)+n1*n2)$

t is number of conditions tags

K is the number of tags (Key in the BST)

n1 is number of photos in previous tag

n2 number of photos in the current tag

- Comparing the two methods, for larger database and when they are many unique tags `invAlbum.getPhotos()` performance is better.

6. Conclusion

In this project, we successfully implemented a photo management system first without using BST and then we use BST with inverted index idea .at the end we compared the time complexity of the two methods, which showed that using BST is better. However, Despite the improved efficiency of the second method, it has limitations. while the use of a BST is efficient for balanced trees, may degrade in performance if the tree becomes unbalanced.